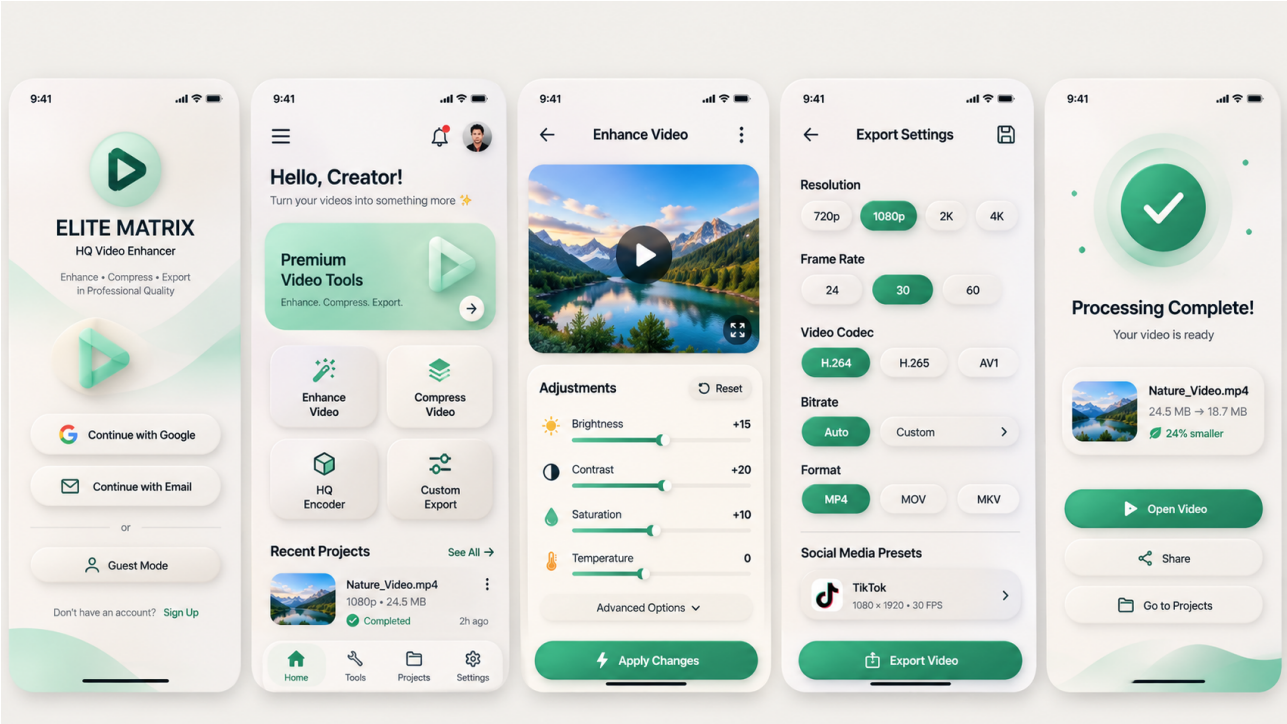
ELITE MATRIX

HQ VIDEO ENHANCER

Complete Product Requirements Document

Exact UI reference implementation • Authentication redesign • Android architecture • FFmpeg • Firebase • Cloud Run

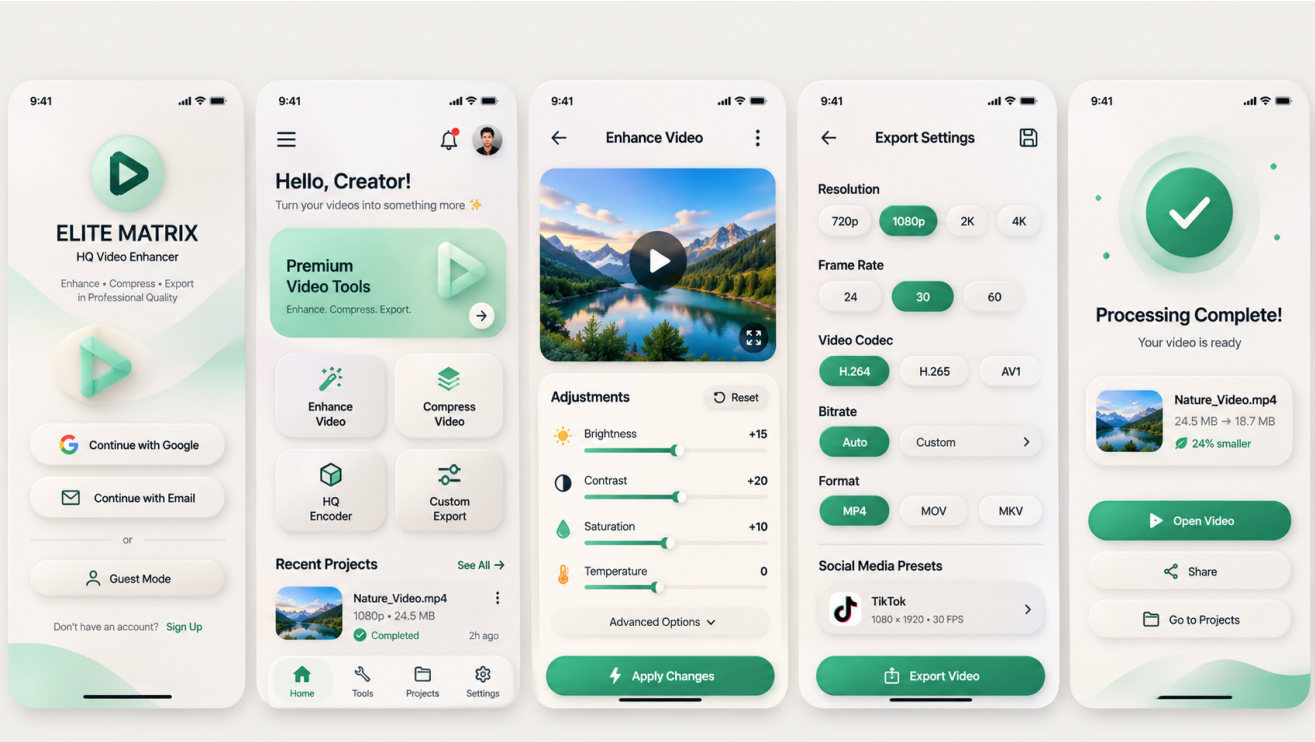


Visual direction: warm ivory surfaces, soft mint accents, restrained green actions, tactile neumorphic depth, crisp typography and compact native Android layouts.

Platform: Native Android • Kotlin • Jetpack Compose • Firebase • FFmpeg • Google Cloud Run

1. Mandatory visual reference and implementation rule

The attached five-screen ELITE MATRIX mockup is the authoritative visual reference for the first implementation. The development agent must reproduce its screen structure, visual hierarchy, spacing, component proportions, colors, typography, icon placement, corner radii, shadows, navigation and content density as closely as practical in native Jetpack Compose.



Five screens shown in the reference

Screen	Required visual and functional content
1. Login / welcome	ELITE MATRIX logo and name, short product description, Continue with Google, Continue with Email, Guest Mode and Sign Up.
2. Home dashboard	Greeting/header, notification and profile actions, primary Enhance/Compress/HQ Encoder/Custom Export tiles, recent projects, quick tools and compact bottom navigation.
3. Enhance Video	Video preview, playback controls, adjustment sliders, reset action, advanced options and Apply Changes.
4. Export Settings	Resolution, FPS, codec, bitrate, format, social export presets and Export Video action.
5. Processing complete	Clear success state, real output filename, original/output sizes, actual savings, Open Video, Share and Go to Projects.

Do not redesign these screens into a different visual concept. Do not replace the reference with generic gradient cards, oversized empty hero sections, excessive glass effects, dashboard clutter or an unrelated template. Improvements are allowed only when required for accessibility, Android system behavior or functional clarity, while preserving the reference's design language.

2. Product definition

ELITE MATRIX is a native Android video enhancement, compression, encoding and export application. It gives creators guided workflows and advanced manual control while maintaining a simple, consistent interface. All processing, authentication, storage, progress and history must be real.

Product goals

- Produce valid playable video outputs using genuine encoding and filter operations.
- Offer quality-focused export profiles and transparent compression trade-offs.
- Support manual resolution, frame rate, bitrate, codec, format, audio and quality controls.

- Support local processing for suitable workloads and cloud processing for resource-intensive jobs.
- Provide secure Google and email authentication, plus an optional local guest mode.
- Persist preferences, presets, jobs and export history accurately.
- Match the approved five-screen neumorphic mockup as the application's visual baseline.

Non-goals and truthful claims

- Do not claim to bypass TikTok, Instagram or YouTube server-side transcoding.
- Do not claim to recover detail that is absent from the source video.
- Do not call basic sharpening or resizing AI enhancement.
- Do not promise unlimited processing regardless of device, cloud, storage or cost constraints.

3. Design system — soft modern neumorphism

Visual language

- Use warm ivory/off-white as the default canvas, deep charcoal-green text and restrained mint/emerald accents.
- Use soft raised and inset surfaces with subtle paired light/dark shadows; avoid heavy drop shadows and exaggerated embossing.
- Use neumorphism selectively for primary tiles, pill controls, sliders and selected states. Text and critical controls must retain strong contrast.
- Use simple consistent line icons; avoid emoji as functional icons.
- Use compact typography, clear grouping and generous but efficient spacing.
- Use subtle state transitions and tactile press feedback without decorative animation overload.

Design tokens

Token	Direction
Canvas	Warm ivory / very light neutral, matching the reference.
Raised surface	Slightly lighter than canvas, with soft upper-left highlight and lower-right shadow.
Inset surface	Recessed neutral surface for sliders, fields and selected input wells.
Accent	Mint / emerald green for primary actions, active states and success.
Text	Deep charcoal-green primary; muted grey-green secondary.
Radius	Consistent rounded cards and pill-shaped controls; avoid mixing unrelated radii.
Elevation	Low-contrast, short-distance shadows; no floating-card overload.
Typography	Native Android-friendly sans-serif; clear title/body/label scale.

Themes and appearance

- Default theme must reproduce the warm ivory and mint mockup.
- Provide additional coordinated pastel themes: Soft Mint, Cream, Lavender, Sky, Peach and Sand.
- Provide Graphite and Midnight dark themes, plus Light, Dark and Follow System modes.
- Persist theme selection locally and apply it consistently to every screen, dialog, bottom sheet and system-bar treatment.
- Ensure accessible contrast; neumorphic shadows must never be the only indicator of selection or state.

4. Login and onboarding — rewritten requirements

The first screen must follow the supplied mockup: a quiet branded welcome screen with a centered ELITE MATRIX mark, product name, concise purpose statement and three clear entry paths. Authentication must use real Firebase Authentication; the screen must not pretend a login succeeded.

Login screen layout

- Top/hero area: circular or softly raised app mark, ELITE MATRIX title and 'HQ Video Enhancer' subtitle.
- Short value statement: 'Enhance • Compress • Export in Professional Quality'.
- Primary outlined/raised action: Continue with Google, with the official Google mark.
- Secondary action: Continue with Email.
- Divider with 'or'.
- Guest Mode as a clearly secondary action.
- Footer: 'Don't have an account? Sign Up'.
- Keep the screen compact and vertically balanced; adapt to smaller devices and keyboard visibility.

Google login flow

- Use Android Credential Manager with the supported Google sign-in option and Firebase Authentication credential exchange.
- Configure the correct application ID, SHA fingerprints, OAuth client and Firebase project.
- On tap, show a real loading state; launch account selection; handle cancellation, no account, network errors and credential failures.
- On success, obtain the Firebase-authenticated user and route to Home only after authentication is confirmed.
- Persist session through Firebase Auth; support sign-out and account switching.
- Never simulate a Google account picker or hardcode a successful user.

Email login and registration flow

- Continue with Email opens a dedicated email sign-in screen with email and password fields, password visibility toggle, validation, Sign In and Forgot Password.
- Sign Up opens a registration screen with name (if required by product), email, password and confirm password.
- Validate email syntax, password policy and matching passwords before sending requests.
- Use Firebase createUserWithEmailAndPassword and signInWithEmailAndPassword.
- Send email verification and clearly show verification state; define whether unverified accounts can use limited local features.
- Forgot Password sends a Firebase password reset email and shows a non-sensitive confirmation.
- Handle duplicate email, invalid credentials, weak password, disabled account, rate limits and network failures with actionable messages.
- Do not reveal whether an email is registered in password-reset responses.

Guest mode

- Guest Mode must be an intentional local-only experience, not a fake account.
- Allow local import, preview and supported on-device processing without Firebase account creation.
- Clearly label cloud processing, cloud backup and cross-device synchronization as sign-in-required.
- If anonymous Firebase Auth is implemented, explain how guest data is handled and provide a secure account-linking flow to Google/email without losing local work.
- Do not silently upload private videos while in guest mode.

Authentication acceptance tests

- Google success, cancellation, unavailable account, network loss and token failure.
- Email registration, sign-in, invalid credentials, weak password, verification and reset.
- Session persistence after process death and app restart.
- Guest local workflow and sign-in upgrade/linking behavior.
- Logout, account switching and account deletion.

5. Screen-by-screen functional specification

Screen 1 — Welcome / Login

Match the mockup's logo placement, title hierarchy, concise descriptor, Google button, email button, divider, Guest Mode and Sign Up footer. Buttons must be accessible, show loading/errors and route to real flows.

Screen 2 — Home dashboard

- Header: menu/navigation affordance, ELITE MATRIX brand, notification action and user avatar.
- Greeting and short helper line.
- Four primary neumorphic action tiles: Enhance Video, Compress Video, HQ Encoder and Custom Export.
- Recent Projects section populated only from real local/cloud job records.
- Quick Tools: Info, Preset, Converter and Calculator; each must open a working feature.
- Processing queue preview showing actual task state and measured progress.
- Bottom navigation: Home, Tools, Projects and Settings.
- Empty state for first-time users with a clear Import Video action; never seed fake recent projects.

Screen 3 — Enhance Video

- Video preview with thumbnail/frame, play/pause, duration and seek controls using a real media player.
- Adjustment controls: brightness, contrast, saturation and temperature, with numeric values and live preview where feasible.
- Reset individual/all adjustments.
- Advanced Options for supported filters such as gamma, tint, highlights/shadows, sharpening, denoising, scaling, crop and rotation.
- Apply Changes creates a real processing job with the exact selected filter graph.
- For expensive processing, explain whether it will run locally or in the cloud and request consent before upload.

Screen 4 — Export Settings

- Resolution selector: original, common presets and validated custom dimensions.
- Frame rate: original and supported values; clearly distinguish frame-rate conversion from motion interpolation.
- Codec: H.264, H.265 and AV1 only if the selected build supports them.
- Bitrate: Auto and Custom; show units and validate plausible ranges.
- Format/container: only show combinations supported by the build and stream configuration.
- Social presets for TikTok, Instagram Reels and YouTube Shorts; label as recommended profiles, not official guarantees.
- Expandable advanced controls: CRF, rate control, encoder preset, profile/level, keyframe interval, pixel format, audio codec/bitrate/sample rate, color handling and metadata options.
- Export Video validates settings and starts a real job.

Screen 5 — Processing complete

- Show only after output validation and successful storage.
- Show real filename, original size, actual output size, actual percentage saved (or size increase), output resolution and duration.
- Open Video launches a real playback screen or compatible player.
- Share invokes Android Sharesheet with the actual output URI and required URI permissions.
- Go to Projects opens the real export record.
- For active jobs, use a separate processing state with genuine upload/worker/download progress; never display a fabricated completion percentage.

6. Full feature requirements

Video import and inspector

- Use Android Photo Picker and Storage Access Framework; retain URI permissions when needed.
- Support common containers only when the selected FFmpeg build can demux/decode them.
- Use FFprobe or equivalent to inspect duration, dimensions, aspect ratio, FPS, bitrate, video/audio codecs, pixel format, rotation, stream count and color/HDR metadata.
- Handle VFR footage, rotated clips, no-audio files, multiple streams, corrupt media and unsupported codecs.

Enhancement

- Brightness, contrast, saturation, gamma, temperature/tint and supported tonal controls.
- Sharpening, denoising, high-quality scaling, deinterlacing, color correction, crop and rotation.
- Optional stabilization and AI super-resolution only when a real model/library, hardware requirements, licensing and resource behavior are implemented.
- Before/after preview must reflect the actual configured processing where technically feasible.

Compression

- Smart, Balanced, High Quality, Maximum Compression and Custom modes.
- Custom target size, bitrate, resolution, FPS, codec, audio settings, CRF, encoder preset and supported rate-control modes.
- Estimate target size using duration, audio bitrate and container overhead; report actual output size after encoding.
- Use two-pass or constrained bitrate where supported; explain that exact file size is not guaranteed.

HQ Encoder

- Presets: HQ Standard, HQ High Detail, HQ Social Media, HQ TikTok, HQ Instagram Reels, HQ YouTube Shorts and Custom.
- Presets are editable starting configurations, not secret platform bypasses.
- Prefer avoiding unnecessary repeated lossy encodes; preserve source FPS and compatible color/metadata when requested.
- Provide an export/transfer workflow for desktop upload where appropriate.

Presets, projects and history

- Create, rename, edit, duplicate, delete and restore presets.
- Store structured local data in Room/DataStore; optionally sync user-owned presets and metadata to Firestore.
- Track jobs through waiting, uploading, queued, processing, downloading, completed, failed and cancelled.
- Retry eligible failures; cancellation and pause/resume must reflect actual worker capabilities.
- History must store actual input/output metadata, settings, timestamps, status and output location.

7. Technical architecture

Android stack

Layer	Required implementation
Language/UI	Kotlin, Jetpack Compose, Material 3 with custom neumorphic components.
Architecture	MVVM + Clean Architecture; modular feature packages.
DI	Hilt.
Local data	Room for jobs/history; DataStore for preferences.
Async/background	Coroutines, Flow, WorkManager for suitable upload/download work.
Playback	AndroidX Media3 ExoPlayer.
Authentication	Firebase Auth + Android Credential Manager.
Media engine	Maintained Android FFmpeg distribution or controlled NDK build; FFprobe.
Cloud	Firebase Auth, Firestore, Cloud Storage, App Check, FCM; Cloud Run and Cloud Tasks.

Suggested package structure

`com.elitematrix.app`
core/design, core/media, core/network, core/database, core/permissions
data/auth, data/local, data/remote, data/repository
domain/entities, domain/repositories, domain/usecases
feature/onboarding, feature/auth, feature/home, feature/enhance, feature/compress, feature/encoder, feature/export, feature/inspector, feature/presets, feature/queue, feature/projects, feature/settings
navigation, MainActivity.kt

FFmpeg integration rules

- Do not select an abandoned wrapper without reviewing maintenance and ABI support.
- Verify arm64-v8a and any additional target ABIs, encoder/filter availability, hardware acceleration and Android NDK compatibility.
- Review FFmpeg and bundled codec licensing (including LGPL/GPL implications) before distributing binaries.
- Do not assume all codecs, filters, pixel formats or hardware encoders are available on every device.
- Run media work off the main thread; stream input/output and bound concurrency.
- Build a processing abstraction so local and cloud workers consume the same validated job configuration.

8. Firebase and Google Cloud design

Responsibilities

- Firebase Auth: identity and sessions.
- Firestore: user profile, settings, presets, job metadata and export metadata.
- Cloud Storage: private source and output video objects.
- Cloud Run service: authenticated job creation/status API.
- Cloud Tasks: durable job dispatch, retry and rate coordination.
- Cloud Run Jobs or worker service: FFmpeg/FFprobe execution for long-running workloads.
- Firebase Cloud Messaging: job completion/failure notifications where enabled.

Suggested Firestore model

Document path	Contents
users/{uid}	Profile and account metadata.
users/{uid}/presets/{presetId}	User-owned validated preset.
users/{uid}/jobs/{jobId}	Operation, configuration, status, progress, timestamps and error code.
users/{uid}/exports/{exportId}	Actual output metadata and object reference.
users/{uid}/settings/preferences	Theme, appearance and user preferences.

Cloud processing sequence

Android → Firebase Authentication → backend API → Cloud Tasks → Cloud Run worker/job → FFprobe validation → FFmpeg processing → output validation → private Cloud Storage → Firestore status update → authorized Android download.

- Verify Firebase ID token and ownership on every protected request.
- Validate source object, size, duration, codec and all requested parameters before dispatch.
- Use private buckets, short-lived signed URLs, least-privilege service identities and Secret Manager.
- Use idempotency keys, retry policies, per-user quotas, request limits and cost controls.
- Track real progress from upload bytes, FFmpeg progress output and download bytes; if precise progress is unavailable, show indeterminate progress rather than invented percentages.
- Clean temporary files and define retention policies for source and output objects.

9. Local/cloud routing and privacy

Route	Appropriate workloads
Local	Metadata inspection, lightweight conversion, simple filters and suitable short clips.
Cloud	Large files, demanding filters, high-resolution jobs and optional AI inference.
Auto	Choose based on file size, codec, requested filters, device capability, network, user preference and cloud availability.

- Ask explicit consent before sending private video to cloud storage/processing.
- Offer local-only processing where supported.
- Use Android's least-privilege media access patterns.
- Never log media content, credentials, access tokens or signed URLs.
- Provide user-controlled deletion and clear account/data deletion behavior.

10. Reliability, performance and error handling

- Never run encoding on the UI thread; expose state via ViewModel and Flow.
- Avoid loading full video files into memory; use file descriptors/streams and temporary files.
- Handle insufficient storage, process death, URI expiry, network interruption and cloud worker failure.
- Persist enough job state to recover or accurately mark an interrupted task after restart.
- Do not claim FFmpeg can pause at arbitrary points; implement cancellation and retry according to actual capabilities.
- Provide readable error messages plus diagnostic codes for unsupported codec, invalid settings, corrupt source, storage, network, authentication, quota and processing failures.

11. Security and data rules

- No service-account JSON or private backend secret in the APK or source repository.
- Firestore and Storage rules must enforce user ownership and deny public access by default.

- Validate all server-side job parameters independently; never trust client-supplied paths or status.
- Use App Check, rate limiting, quotas and abuse monitoring.
- Separate development, staging and production Firebase/GCP projects.
- Set budget alerts and lifecycle cleanup for cloud storage and processing resources.

12. Testing and acceptance criteria

Required tests

- Authentication: Google, email, reset, verification, session restore, guest mode and account deletion.
- Media: H.264/H.265 and supported codecs, VFR, portrait rotation, no-audio files, HDR, multiple streams, corrupt and large files.
- Output: playback, audio sync, dimensions, FPS, bitrate, codec, metadata, color behavior and actual file size.
- Cloud: token validation, cross-user denial, expired URL, duplicate request, retry, worker failure, quota and cleanup.
- UI: all five reference screens, theme variants, dark/light mode, small screens, keyboard, accessibility and error/empty states.

Release gate